Functional architecture

English version of ../../architecture/functional.md. The diagram shows the KAM user path (5 steps: authorization, browsing, filtering, status update, report) and the backend services that serve it. The Mermaid diagram renders on GitHub/GitLab; when needed it is exported to PNG/PDF for a presentation.

The same model in ArchiMate 3 is `functional.archimate` (opens in Archi), with a ready PDF version. The Mermaid source for rendering is `functional.mmd`.

The user path and the services in Archi

```
flowchart TD
    subgraph BIZ["Business layer: the user path (section 2 of the specification)"]
        direction LR
        S1["Step 1. Authorization<br/>login/password or Keycloak"]
        S2["Step 2. Browsing interactions<br/>KAM — own universities, executive — all"]
        S3["Step 3. Filtering<br/>university, product, direction, B2B/B2C"]
        S4["Step 4. Status update<br/>comment + files + stage change"]
        S5["Step 5. Report<br/>summary and export XLSX/XLS/PDF/JSON"]
        S1 --> S2 --> S3 --> S4 --> S5
    end
    subgraph APP["Application layer: services"]
        AUTH["Auth Service (Keycloak)"]
        BOARD["Board Service (Kanban)"]
        CARD["Interaction Service (cards)"]
        REPORT["Report Service (reports)"]
        FILES["File Service (MinIO / S3)"]
        INTEG["Integration Service (LMS / CMS)"]
        AI["AI Service (GigaChat, FT-6)"]
    end
    OBJ[("Interaction<br/>university + product")]
    S1 --> AUTH
    S2 --> BOARD
    S2 --> CARD
    S3 --> BOARD
    S4 --> CARD
    S4 --> FILES
    S4 -.->|on request| AI
    S5 --> REPORT
    S5 --> INTEG
    S4 --> OBJ
    S5 --> OBJ
```

Operating cycle: import → board → card → report

```
flowchart LR
    start([KAM signs in]) --> s1[Step 1.<br/>Data import<br/>XLS / XLSX / JSON]
    s1 --> s2[Step 2.<br/>Working with the board<br/>Kanban by stage]
    s2 --> s3[Step 3.<br/>Interaction card<br/>contract, tasks, files]
    s3 --> s4[Step 4.<br/>Moving through stages<br/>recording the outcome]
    s4 --> s5[Step 5.<br/>Report and export<br/>XLSX / XLS / PDF / JSON / PNG]

    s2 -.->|filters| f1[By university, product, scope B2B/B2C]
    s3 -.->|notifications| f2[Stalled requests → to the executive]
    s5 -.->|audit| f3[Action log + CSV]
```

Operations and supporting services

Cycle stage	Screen	Backend service	Key entities
1. Data import	`DirectoryPage`	`services/excel_import.py`, `services/import_report.py`	`University`, `ITProduct`, `Interaction`
2. Board	`BoardPage`	`api/interactions.py` (list, board)	`Interaction`, `WorkflowStageRef`
3. Card	`InteractionDrawer`	`api/interactions.py`, `api/files.py`	`Action`, `Comment`, `FileObject`
4. Stage change	board / card	`api/interactions.py` (stage update)	`Interaction.stage_id`
5. Reports and export	`ReportPage`, `AuditLogPage`	`api/reports.py`, `api/audit.py`, `services/excel_export.py`	`ActionLog`, aggregates

Functional blocks

```
flowchart TB
    subgraph Presentation["Presentation (React SPA)"]
        board[Kanban board]
        card[Interaction card]
        report[Reports and charts]
        dirs[Directories and Wiki]
        audit_ui[Audit log]
    end

    subgraph Application["Application (FastAPI)"]
        interactions[Interactions API]
        catalogs[Catalogs / Import API]
        reports_api[Reports / Export API]
        audit_api[Audit API]
        authz[Auth + RBAC]
    end

    subgraph Domain["Domain / Infrastructure"]
        workflow[Workflow: B2B/B2C stages]
        cache[Report cache Redis/KeyDB]
        scheduler[Notification scheduler]
        storage[File storage S3/MinIO]
        gigachat[GigaChat: card summary]
        db[(PostgreSQL)]
    end

    board --> interactions
    card --> interactions
    dirs --> catalogs
    report --> reports_api
    audit_ui --> audit_api
    interactions --> authz
    catalogs --> authz
    reports_api --> authz
    audit_api --> authz
    interactions --> workflow --> db
    reports_api --> cache --> db
    catalogs --> storage
    scheduler --> db
    card -.->|summary, FT-6| gigachat
```

The key business logic is a single workflow without versions: a stage change applies immediately to all active requests, and deleting a stage moves the tasks to a neighbouring one so that work never stops.

GigaChat is called only on an explicit user request from the card and takes no part in the main scenario: without a key the "Summary" tab reports that the service is not configured. Details — `docs/AI.md`.